

Interactive Genetic Mapping: Three-Point Cross Analysis

Table of contents

1 Student Instructions	1
1.1 Overview	1
1.2 Part 1: Recombination Frequency vs Map Distance	1
1.3 Part 2: Crossovers vs Recombinants	2
1.4 Part 3: Three-Point Cross — Determining Gene Order	2
1.5 Summary of Key Concepts	3
2 Code	4
2.1 Interactive Genetic Mapping: Three-Point Cross Analysis	4

1 Student Instructions

1.1 Overview

This notebook takes you through three core concepts in genetic mapping: how recombination frequency relates to map distance, why double crossovers are a problem for accurate mapping, and how to determine gene order from a three-point testcross. Work through the parts in order — each builds on the previous one.

1.2 Part 1: Recombination Frequency vs Map Distance

What is this?

When two genes are linked on the same chromosome, we measure how often they recombine during meiosis. This **recombination frequency (RF)** is our raw observable — but it is not the same as map distance. **Haldane's mapping function** converts RF to map distance (centiMorgans, cM), correcting for double crossovers that go undetected because they restore the parental combination.

The two plots explained

- **Left plot:** Shows how Haldane's curve (blue) diverges from the simple linear expectation (red dashed). At short distances they agree, but as RF increases toward 50%, Haldane assigns increasingly larger map distances than RF alone would suggest.
- **Right plot:** Shows the *extra* map distance added by the Haldane correction — i.e., the distance hidden in undetected double crossovers. This correction grows rapidly and approaches infinity as $RF \rightarrow 50\%$, because genes that appear completely unlinked ($RF = 50\%$) could still be on the same chromosome but very far apart.

How to use it

1. Move the **RF slider** and observe how both plots update
2. At low RF (< 0.10), notice the two methods give nearly identical results — double crossovers are rare when genes are close
3. At $RF > 0.30$, notice the rapidly growing divergence — Haldane adds substantial distance that RF alone misses
4. At $RF = 0.50$, the curve asymptotes to infinity — RF can never exceed 50% even for very distant genes

Key questions to answer

- At what RF does the Haldane correction start to become substantial?

- Why can RF never exceed 50%, even for genes very far apart?
 - Why does the right-hand curve approach infinity rather than levelling off?
-

1.3 Part 2: Crossovers vs Recombinants

What is this?

This part illustrates the key problem with using RF as a direct measure of distance: **double crossovers restore parental combinations and are invisible to us**. A meiosis with two crossovers between two loci looks exactly like a meiosis with no crossovers when we score the offspring.

The two plots explained

- **Left plot:** Shows the distribution of meioses with 0, 1, 2 or 3+ crossovers. As map distance increases, double and triple crossovers become more common.
- **Right plot:** Shows the consequence — progeny are classified as either **parental** (even number of crossovers, including zero) or **recombinant** (odd number of crossovers). The recombinant class never exceeds 50% because every new double crossover moves an individual back into the parental class.

How to use it

1. Set map distance to **10 cM** — note that almost all meioses have zero crossovers, and single crossovers dominate the recombinant class
2. Increase to **40 cM** — double crossovers become much more frequent; notice how recombinant % grows more slowly than the map distance
3. Increase to **80–100 cM** — double crossovers are now very common and the recombinant class approaches but never reaches 50%
4. This is exactly why Haldane's function is necessary: RF systematically underestimates distance, and the underestimation grows with distance

Key questions to answer

- At 20 cM map distance, what proportion of meioses have a double crossover? What does this look like in the offspring?
 - Why do recombinants plateau near 50% and not continue increasing?
 - How does this simulation explain the shape of the Haldane curve you saw in Part 1?
-

1.4 Part 3: Three-Point Cross — Determining Gene Order

What is this?

A **three-point testcross** is the classical method for simultaneously mapping three linked genes. A triple heterozygote (e.g., ABC/abc) is crossed to a triple recessive (abc/abc), and the offspring are classified into eight genotypic classes. The relative frequencies of these classes reveal both the distances between genes and their **order** on the chromosome.

The four progeny classes

Each class comes in a reciprocal pair (two rows with equal frequency), giving eight rows total:

Class	What happened	Expected frequency
Parental	No crossover in either interval	Most frequent
SCO Region I	Single crossover between genes 1 and 2	Intermediate
SCO Region II	Single crossover between genes 2 and 3	Intermediate
Double CO	Crossover in BOTH intervals	Least frequent

How to determine gene order – the key logic

The **double crossover class is the detective tool for gene order**. Compare the parental class with the double crossover class:

1. **Identify the parental classes** (most frequent pair) – these tell you which alleles were on the same chromosome at the start
2. **Identify the DCO classes** (least frequent pair) – a double crossover in both flanking intervals affects only the *middle* gene
3. **Find which allele switched** between parental and DCO – that gene is in the **middle position**
4. The two outer genes stay together in both parental and DCO classes

How to use it

1. Start with gene order **ABC**, distances 10 cM and 15 cM
2. Confirm the parental class (ABC and abc) is most frequent
3. Confirm the DCO class (A b C and a B c) is least frequent – note only B/b has switched relative to the parentals
4. Now change gene order to **ACB** – observe how the SCO classes change, but the DCO class still reveals the middle gene
5. Try **unequal distances** (e.g., 5 cM and 25 cM) – notice how SCO Region II becomes much more frequent than SCO Region I, reflecting the longer second interval
6. The bar chart summarises all classes – frequency always follows: **Parental > SCO (larger interval) > SCO (smaller interval) > DCO**

Key questions to answer

- For gene order ACB with distances 10 cM and 20 cM: which allele switches between parental and DCO classes? What does this confirm?
- Why is the DCO class always the least frequent, regardless of the distances involved?
- If you were given only the offspring counts (without being told the gene order), how would you work out which gene is in the middle?
- How would you calculate the map distance between genes 1 and 2 from the progeny class counts?

1.5 Summary of Key Concepts

Concept	Key point
Recombination frequency (RF)	Raw observable; max = 50%; underestimates true distance

Concept	Key point
Haldane's mapping function	Corrects RF for undetected double crossovers; gives cM
Double crossover	Restores parental combination; makes RF an underestimate
centiMorgan (cM)	Unit of map distance; 1 cM $\approx$ 1% recombination for short distances
Gene order	Determined by DCO class — the middle gene switches alleles relative to parentals
Interference	Tendency of one crossover to inhibit nearby crossovers; reduces DCO frequency

2 Code

2.1 Interactive Genetic Mapping: Three-Point Cross Analysis

Learn how to: - Calculate recombination frequencies from trihybrid cross data - Convert recombination frequency to map distance (centiMorgans) - Apply Haldane's mapping function to account for double crossovers - Determine gene order from recombinant class frequencies - Visualize the relationship between crossovers and recombination

```
#@title Utility functions
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from ipywidgets import interact, Dropdown, FloatSlider

def haldane_mapping_function(RF):
    """
    Convert recombination frequency to map distance using Haldane's function.
    m = -50 * ln(1 - 2*RF)
    Asymptotes to infinity as RF → 0.5 (genes on opposite chromosome ends).
    """
    if RF >= 0.5:
        return np.inf # Mathematically correct: map distance → ∞ as RF → 0.5
    return -50 * np.log(1 - 2 * RF)

def inverse_haldane(map_distance):
    """
    Convert map distance back to recombination frequency.
    RF = 0.5 * (1 - e^(-m/50))
    """
    return 0.5 * (1 - np.exp(-map_distance / 50))

def visualize_rf_vs_map_distance(selected_RF):
    # Use values strictly below 0.5 to avoid infinity issues in plotting
    RF_values = np.linspace(0, 0.499, 300)
```

```

map_distances = np.array([-50 * np.log(1 - 2 * rf) for rf in RF_values])
differences = map_distances - RF_values * 100

selected_map = haldane_mapping_function(selected_RF)
expected_linear = selected_RF * 100

print(f"Selected recombination frequency: {selected_RF:.3f} ({selected_RF*100:.1f}%)")
print("\nIf there were NO double crossovers:")
print(f"  Map distance would be: {expected_linear:.2f} cM")
print("\nUsing Haldane's mapping function (accounts for double crossovers):")
print(f"  Map distance is: {selected_map:.2f} cM")
print(f"\nDifference: {selected_map - expected_linear:.2f} cM")
print("This difference increases as genes get farther apart,")
print("approaching infinity as RF → 50% (unlinked genes).\n")

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

# Left: RF vs map distance curve
ax1.plot(RF_values * 100, map_distances, 'b-', linewidth=2, label='Haldane function')
ax1.plot([0, 50], [0, 50], 'r--', linewidth=2, label='No double crossovers (linear)')
ax1.scatter([selected_RF * 100], [selected_map], color='red', s=100, zorder=5,
            label='Your selection')
ax1.axhline(selected_map, color='gray', linestyle=':', alpha=0.5)
ax1.axvline(selected_RF * 100, color='gray', linestyle=':', alpha=0.5)
ax1.set_xlabel('Recombination Frequency (%)', fontsize=12)
ax1.set_ylabel('Map Distance (cM)', fontsize=12)
ax1.set_title('Recombination Frequency vs Map Distance', fontsize=13, weight='bold')
ax1.legend()
ax1.grid(alpha=0.3)
ax1.set_xlim(0, 50)
ax1.set_ylim(0, 75)

# Right: Difference curve – correctly asymptotes as RF → 50%
ax2.plot(RF_values * 100, differences, 'purple', linewidth=2)
if selected_RF < 0.499:
    ax2.scatter([selected_RF * 100], [selected_map - expected_linear],
                color='red', s=100, zorder=5)
ax2.axhline(0, color='black', linestyle='-', linewidth=0.5)
ax2.set_xlabel('Recombination Frequency (%)', fontsize=12)
ax2.set_ylabel('Difference (cM)', fontsize=12)
ax2.set_title('Extra distance from undetected double crossovers', fontsize=13,
weight='bold')
ax2.grid(alpha=0.3)
ax2.set_xlim(0, 50)
ax2.set_ylim(0, max(differences[np.isfinite(differences)]) * 1.1)

# Add asymptote annotation
ax2.annotate('→ ∞ as RF → 50%\n(completely unlinked)',
            xy=(48, max(differences) * 0.85),
            fontsize=10, color='purple', ha='right')

```

```

plt.tight_layout()
plt.show()

def simulate_crossovers(num_individuals, crossover_prob):
    """
    Simulate meiotic events with crossovers between two loci.
    """
    # For each individual, determine number of crossovers
    crossover_counts = np.random.poisson(crossover_prob, num_individuals)

    # Odd number of crossovers = recombinant
    # Even number of crossovers = parental
    recombinants = crossover_counts % 2 == 1

    results = {
        'total': num_individuals,
        'no_crossover': np.sum(crossover_counts == 0),
        'single_crossover': np.sum(crossover_counts == 1),
        'double_crossover': np.sum(crossover_counts == 2),
        'triple_plus': np.sum(crossover_counts >= 3),
        'recombinants': np.sum(recombinants),
        'parentals': np.sum(~recombinants),
        'total_crossovers': np.sum(crossover_counts),
        'avg_crossovers_per_meiosis': np.mean(crossover_counts)
    }

    return results

def visualize_crossovers_vs_recombinants(map_distance_input):
    """
    Show how crossovers relate to observed recombinants.
    """
    # Convert map distance to expected crossovers per meiosis
    crossover_rate = map_distance_input / 100

    # Simulate
    np.random.seed(42)
    results = simulate_crossovers(10000, crossover_rate)

    observed_RF = results['recombinants'] / results['total']

    print(f"Map distance: {map_distance_input} cM")
    print(f"Expected crossovers per meiosis: {crossover_rate:.3f}")
    print("\nResults from 10,000 simulated meioses:")
    print(f"  No crossover: {results['no_crossover']:,} ({results['no_crossover']/results['total']*100:.1f}%)")
    print(f"  Single crossover: {results['single_crossover']:,} ({results['single_crossover']/results['total']*100:.1f}%)")
    print(f"  Double crossover: {results['double_crossover']:,} ({results['double_crossover']/results['total']*100:.1f}%)")
    print(f"  Triple+ crossover: {results['triple_plus']:,} ({results['triple_plus']/results['total']*100:.1f}%)")

```

```

results['total']*100:.1f}%")
    print(f"\nTotal crossovers: {results['total_crossovers']:,}")
    print(f"Average crossovers per meiosis: {results['avg_crossovers_per_meiosis']:.3f}")
    print(f"\nRecombinants observed: {results['recombinants']:,} ({observed_RF*100:.2f}%)")
    print(f"Parental types: {results['parentals']:,} ({results['parentals']/
results['total']*100:.2f}%)")
    print("\nKey insight: Even crossovers restore parental → RF underestimates crossover
frequency!\n")

# Visualize
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

# Crossover distribution
categories = ['None', 'Single', 'Double', 'Triple+']
counts = [results['no_crossover'], results['single_crossover'],
          results['double_crossover'], results['triple_plus']]
colors = ['lightblue', 'lightcoral', 'lightgreen', 'plum']

bars = ax1.bar(categories, counts, color=colors, edgecolor='black', linewidth=1.5)
ax1.set_ylabel('Number of meioses', fontsize=12)
ax1.set_title(f'Crossover Distribution (map distance = {map_distance_input} cM)',
              fontsize=13, weight='bold')
ax1.grid(axis='y', alpha=0.3)

for bar, count in zip(bars, counts):
    height = bar.get_height()
    ax1.text(bar.get_x() + bar.get_width()/2., height,
             f'{count:,}\n({count/results["total"]*100:.1f}%)',
             ha='center', va='bottom', fontsize=10)

# Parental vs recombinant
outcome_cats = ['Parental\n(even crossovers)', 'Recombinant\n(odd crossovers)']
outcome_counts = [results['parentals'], results['recombinants']]
outcome_colors = ['skyblue', 'salmon']

bars2 = ax2.bar(outcome_cats, outcome_counts, color=outcome_colors,
                edgecolor='black', linewidth=1.5)
ax2.set_ylabel('Number of gametes', fontsize=12)
ax2.set_title('Observable Phenotype', fontsize=13, weight='bold')
ax2.grid(axis='y', alpha=0.3)

for bar, count in zip(bars2, outcome_counts):
    height = bar.get_height()
    ax2.text(bar.get_x() + bar.get_width()/2., height,
             f'{count:,}\n({count/results["total"]*100:.1f}%)',
             ha='center', va='bottom', fontsize=11, weight='bold')

plt.tight_layout()
plt.show()

def generate_three_point_data_correct(gene_order, dist_12, dist_23, sample_size=1000):

```

```

"""
Generate simulated three-point cross data with CORRECT genotype assignments.
gene_order: string like 'ABC' specifying order on chromosome (1-2-3)
dist_12: map distance between first and second gene
dist_23: map distance between second and third gene

Key principle: DCO switches ONLY the middle gene relative to parentals
"""

# Calculate probabilities
RF_12 = inverse_haldane(dist_12)
RF_23 = inverse_haldane(dist_23)

# Approximate class frequencies (no interference)
P_no_CO = (1 - RF_12) * (1 - RF_23)
P_CO_region1 = RF_12 * (1 - RF_23)
P_CO_region2 = (1 - RF_12) * RF_23
P_CO_both = RF_12 * RF_23

gene1, gene2, gene3 = gene_order[0], gene_order[1], gene_order[2]

# Parental chromosomes: all dominant linked or all recessive linked
# Notation: ABC means A+B+C+ (all dominant on same chromosome)
parental_1 = f"{gene1} {gene2} {gene3}"
parental_2 = f"{gene1.lower()} {gene2.lower()} {gene3.lower()}"

# Single crossover region 1 (between gene1 and gene2)
# Swaps relationship between gene1 and (gene2-gene3 unit)
sco1_1 = f"{gene1.lower()} {gene2} {gene3}"
sco1_2 = f"{gene1} {gene2.lower()} {gene3.lower()}"

# Single crossover region 2 (between gene2 and gene3)
# Swaps relationship between (gene1-gene2 unit) and gene3
sco2_1 = f"{gene1} {gene2} {gene3.lower()}"
sco2_2 = f"{gene1.lower()} {gene2.lower()} {gene3}"

# Double crossover (in BOTH regions)
# ONLY the middle gene switches relative to the outer two
dco_1 = f"{gene1} {gene2.lower()} {gene3}"
dco_2 = f"{gene1.lower()} {gene2} {gene3.lower()}"

# Generate counts
counts = {
    'Parental_1': int(sample_size * P_no_CO / 2),
    'Parental_2': int(sample_size * P_no_CO / 2),
    'SCO_region1_1': int(sample_size * P_CO_region1 / 2),
    'SCO_region1_2': int(sample_size * P_CO_region1 / 2),
    'SCO_region2_1': int(sample_size * P_CO_region2 / 2),
    'SCO_region2_2': int(sample_size * P_CO_region2 / 2),
    'DCO_1': int(sample_size * P_CO_both / 2),
    'DCO_2': int(sample_size * P_CO_both / 2),
}

```



```

# Adjust to match sample size
total = sum(counts.values())
counts['Parental_1'] += sample_size - total

genotypes = {
    'Parental_1': parental_1,
    'Parental_2': parental_2,
    'SCO_region1_1': sco1_1,
    'SCO_region1_2': sco1_2,
    'SCO_region2_1': sco2_1,
    'SCO_region2_2': sco2_2,
    'DCO_1': dco_1,
    'DCO_2': dco_2,
}

return counts, genotypes, RF_12, RF_23, gene1, gene2, gene3

def analyze_three_point_cross_correct(gene_order_input, dist_12_input, dist_23_input):
    """
    Interactive three-point cross analyzer with CORRECT logic.
    """
    counts, genotypes, RF_12, RF_23, g1, g2, g3 = generate_three_point_data_correct(
        gene_order_input, dist_12_input, dist_23_input, sample_size=1000
    )

    print(f"Gene order on chromosome: {gene_order_input}")
    print(f"Distance {g1}-{g2}: {dist_12_input} cM → RF = {RF_12:.4f}")
    print(f"Distance {g2}-{g3}: {dist_23_input} cM → RF = {RF_23:.4f}")
    print(f"\nSimulated testcross: {g1} {g2} {g3} / {g1.lower()} {g2.lower()} {g3.lower()}")
    print(f"      {g1.lower()} {g2.lower()} {g3.lower()} / {g1.lower()} {g2.lower()} {g3.lower()}")
    print(f"      {g3.lower()}\n")
    print("(Capital = dominant allele, lowercase = recessive allele)")
    print("(All testcross progeny are heterozygous at recessive parent's loci)\n")

    # Create results table
    data_rows = []
    for key in ['Parental_1', 'Parental_2', 'SCO_region1_1', 'SCO_region1_2',
               'SCO_region2_1', 'SCO_region2_2', 'DCO_1', 'DCO_2']:
        phenotype_class = 'Parental' if 'Parental' in key else \
            (f'SCO Region I ({g1}-{g2})' if 'region1' in key else \
             (f'SCO Region II ({g2}-{g3})' if 'region2' in key else 'Double
CO'))
        data_rows.append({
            'Gamete from heterozygote': genotypes[key],
            'Count': counts[key],
            'Class': phenotype_class
        })

    df = pd.DataFrame(data_rows)

```

```

print("Progeny classes (gamete from heterozygous parent):")
print(df.to_string(index=False))

df_summary = df.groupby('Class')['Count'].sum().reset_index()
df_summary['Percentage'] = (df_summary['Count'] / 1000 * 100).round(1)

print("\nSummary by class:")
print(df_summary.to_string(index=False))

print("\n--- HOW TO DETERMINE GENE ORDER ---")
print(f"1. Identify PARENTAL classes (most frequent): {counts['Parental_1'] + counts['Parental_2']} total")
print(f"    Parental 1: {genotypes['Parental_1']}")
print(f"    Parental 2: {genotypes['Parental_2']}")

print(f"\n2. Identify DOUBLE CROSSOVER classes (least frequent): {counts['DCO_1'] + counts['DCO_2']} total")
print(f"    DCO 1: {genotypes['DCO_1']}")
print(f"    DCO 2: {genotypes['DCO_2']}")

print("\n3. Compare parentals vs DCO to find the middle gene:")
print(f"    • Parental 1: {genotypes['Parental_1']}")
print(f"    • DCO 1:      {genotypes['DCO_1']}")
print(f"    → Gene {g2} switched (was with {g1} and {g3}, now alone)")
print(f"    → {g2} is the MIDDLE gene")

print(f"\n4. Gene order confirmed: {g1}—{g2}—{g3}\n")

# Visualize
fig, ax = plt.subplots(figsize=(12, 6))

classes_order = ['Parental', f'SCO Region I ({g1}-{g2})', f'SCO Region II ({g2}-{g3})', 'Double CO']
class_counts = [df_summary[df_summary['Class'] == c]['Count'].values[0] if len(df_summary[df_summary['Class'] == c]) > 0 else 0 for c in classes_order]
colors_map = {'Parental': 'skyblue', f'SCO Region I ({g1}-{g2})': 'lightcoral', f'SCO Region II ({g2}-{g3})': 'lightgreen', 'Double CO': 'gold'}
colors = [colors_map[c] for c in classes_order]

bars = ax.bar(classes_order, class_counts, color=colors, edgecolor='black', linewidth=2)
ax.set_ylabel('Number of progeny', fontsize=12)
ax.set_title(f'Three-Point Cross Results: {gene_order_input} gene order\n'
             f'({dist_12_input} cM between {g1}-{g2}, {dist_23_input} cM between {g2}-{g3})',
             fontsize=13, weight='bold')
ax.grid(axis='y', alpha=0.3)

for bar, count in zip(bars, class_counts):

```

```

height = bar.get_height()
ax.text(bar.get_x() + bar.get_width()/2., height,
        f'{count}\n({count/10:.1f}%)',
        ha='center', va='bottom', fontsize=11, weight='bold')

plt.tight_layout()
plt.show()

```

Part 1: Recombination Frequency vs Map Distance

Recombination frequency (RF) measures observed recombinants. Map distance (cM) accounts for undetected double crossovers.

```

interact(visualize_rf_vs_map_distance,
        selected_RF=FloatSlider(value=0.15, min=0, max=0.5, step=0.01,
                                description='RF:',
                                style={'description_width': '80px'}))

```

Part 2: Crossovers vs Recombinants

Key concept: double crossovers restore parental combinations, which is why observed RF is lower than actual crossover frequency.

```

interact(visualize_crossovers_vs_recombinants,
        map_distance_input=FloatSlider(value=20, min=5, max=100, step=5,
                                        description='Map dist (cM):',
                                        style={'description_width': '100px'}))

```

Part 3: Three-Point Cross — Determining Gene Order

Use a trihybrid testcross to map three linked genes. Gene order is determined by identifying the double-crossover class.

```

interact(analyze_three_point_cross_correct,
        gene_order_input=Dropdown(options=['ABC', 'ACB', 'BAC'], value='ABC',
                                   description='Gene order:',
                                   style={'description_width': '100px'}),
        dist_12_input=FloatSlider(value=10, min=5, max=30, step=5,
                                   description='Dist 1-2 (cM):',
                                   style={'description_width': '100px'}),
        dist_23_input=FloatSlider(value=15, min=5, max=30, step=5,
                                   description='Dist 2-3 (cM):',
                                   style={'description_width': '100px'}))

```